

Deadlock Freedom

Max Moir

June 1, 2026

1 Question

Hello Chenyi, I wanted to address some confusion I have about the definition of deadlock freedom. It seems that in some papers deadlock freedom is expressed as a liveness property, expressing progress, whereas in others it is a safety property. You have suggested I should treat it as a liveness property, assuming fairness and reasoning about the execution path using temporal logic. In some of the papers I've seen, absence of deadlock is treated as a safety property, where one must just prove that it is impossible to reach a deadlocked state. For example, it is described this way in the following two papers.

In [3], Lamport writes: “*Examples of safety properties are deadlock freedom, where Etern asserts that the program is not in a deadlock state, and mutual exclusion, where Etern asserts that two processes are not both in their critical sections.*” This definition of deadlock freedom describes it differently, in way such that it could be represented simply as an invariant like mutex.

Also, in [4]: “*Note that the absence of deadlock is a safety property, since it implies that a bad state (one in which both processes are waiting) cannot occur. Hence, it can be proved with the method described in Section 4.*” In this paper following definition is also given. “*Absence of deadlock: the program never enters a state in which no further progress is possible.*” These definitions are more limited than the one we talked about, so I'm wondering whether proving deadlock freedom in this way is sufficient.

2 Example papers

Additionally, in these two papers by Hesselink, in which different variants of the Bakery algorithm are formally verified, he proves absence of deadlock by showing deadlock states are not reachable. In the first [1], he states: “*Absence of deadlock means that, when there is a competing process (a process not at 10), there is a competing process that can do a forward step. Of course, absence of deadlock is required only in reachable states. We may therefore use available invariants.*”

Similarly, in another of his papers [2] he writes: “*The transition system is said to be in deadlock iff there are competing threads and no (competing) thread can do a forward step. Absence of deadlock means that deadlock states are not reachable.*” Based on the proofs given in these papers (and some others I've seen), a invariant based approach also seems sufficient.

References

- [1] W. H. Hesselink. Mechanical verification of Lamport's Bakery algorithm. *Science of Computer Programming*, 78(9):1622–1638, Sept. 2013.
- [2] W. H. Hesselink. Correctness and concurrent complexity of the Black-White Bakery Algorithm. *Formal Aspects of Computing*, 28(2):325–341, Apr. 2016.
- [3] L. Lamport and F. B. Schneider. The “Hoare Logic” of CSP, and All That. *ACM Transactions on Programming Languages and Systems*, 6(2):281–296, Apr. 1984.
- [4] S. Owicki and L. Lamport. Proving Liveness Properties of Concurrent Programs. *ACM Transactions on Programming Languages and Systems*, 4(3):455–495, July 1982.